

Ten surprises from numerical linear algebra

Posted on **20 January 2010** by **John**

Here are ten things about numerical linear algebra that you may find surprising if you're not familiar with the field.

1. Numerical linear algebra applies very advanced mathematics to solve problems that can be stated with high school mathematics.
2. Practical applications often require solving enormous systems of equations, millions or even billions of variables.
3. The heart of Google is an enormous linear algebra problem. PageRank is essentially an eigenvalue problem.
4. The efficiency of solving very large systems of equations has benefited at least as much from advances in algorithms as from Moore's law.
5. Many practical problems — optimization, differential equations, signal processing, etc. — boil down to solving linear systems, even when the original problems are non-linear. Finite element software, for example, spends nearly all its time solving linear equations.
6. A system of a million equations can sometimes be solved on an ordinary PC in under a millisecond, depending on the structure of the equations.
7. Iterative methods, methods that in theory require an infinite number of steps to solve a problem, are often faster and more accurate than direct methods, methods that in theory produce an exact answer in a finite number of steps.
8. There are many theorems bounding the error in solutions *produced on real computers*. That is, the theorems don't just bound the error from hypothetical calculations carried out in exact arithmetic but bound the error from arithmetic as carried out in [floating point arithmetic](#) on computer hardware.
9. It is hardly ever necessary to compute the [inverse](#) of a matrix.
10. There is remarkably mature [software](#) for numerical linear algebra. Brilliant people have worked on this software for many years.

More applied linear algebra

- [Don't invert that matrix](#)
- [Applying PageRank to biology](#)
- [Numerical computing consulting](#)

Categories : [Computing](#) [Math](#)

Tags : [Linear algebra](#) [Numerical analysis](#) [Programming](#)

Bookmark the [permalink](#)

Previous Post
[Don't invert that matrix](#)

Next Post
[How to compute the soft maximum](#)

23 thoughts on “Ten surprises from numerical linear algebra”

Sean Devlin

20 January 2010 at 10:53

No love for ATLAS? I thought this was use in favor of lapack nowadays.

<http://math-atlas.sourceforge.net/>

Giles Warrack

20 January 2010 at 11:31

Re No 3, Herbert Wilf (Penn) has a nice handout on the Kendall-Wei algorithm and Google

<http://www.math.upenn.edu/~wilf/>

joevo

20 January 2010 at 12:27

@Sean: LAPACK is built on top of BLAS/ATLAS.

Evan

20 January 2010 at 22:48

So.. you have convinced me to learn Linear Algebra. Do you have any suggestions for books to get started? Thanks for the inspiration!

John

21 January 2010 at 00:11

Evan, there are a lot of linear algebra books out there. I got a review copy of Cheney and Kincaid's [linear algebra book](#) a few weeks ago and I thought it was a nice text. It's easy to read and has a good mix of theory and application.

If you know basic linear algebra and want to jump into numerical linear algebra, I recommend Demmel's [Applied Numerical Linear Algebra](#). Also, there's the classic Matrix Calculations by Golub and Van Loan.

Evan

21 January 2010 at 14:54

John, thanks for your quick reply. I'll definitely check those out!

Matt

24 January 2010 at 06:46

By the way, has anyone here played with GotoBLAS?

Is its performance really as good as the benchmarks on its website suggest or is it just advertisement?

The source code is freely available, though only for research applications (commercial is not free). Here are relevant links:

<http://www.tacc.utexas.edu/resources/software/gotoblasfaq/>

http://en.wikipedia.org/wiki/Kazushige_Goto

<http://www.tacc.utexas.edu/tacc-projects/>

<http://web.tacc.utexas.edu/~kgoto/>

http://www.pathscale.com/building_code/gotoblas.html

Michael Holmes

1 April 2010 at 08:33

John suggested I post this here –

We recently released a MATLAB package called QLA that dramatically accelerates core linear algebra routines on dense matrices. It includes SVD, linear systems, least squares, PCA, etc., and speedups range from 10x to 1,000x+ on benchmarks. The gains come by trading a minuscule bit of accuracy for a mammoth increase in speed.

You can find more details and the free beta version here:

<http://massiveanalytics.com>.

Ron Modesitt

26 July 2010 at 18:46

What a thoroughly interesting blog! Glad I found you, with thanks to Stumble.

Ron

Luis

19 July 2011 at 17:45

Another interesting surprise is that, often, the matrices involved are very sparse.

Books: I love “Matrix algebra useful for statistics” by Searle.

Utkarsh Upadhyay

13 September 2011 at 11:02

Some surprises indeed!

However, a small note for point 4:

The efficiency of solving very large systems of equations has benefited at least as much from advances in algorithms as from Moore’s law.

At least solutions for sparse system of equations have benefited [two orders of magnitude more from algorithmic developments than hardware advancements](#) (slide 7) to a total of *10 orders of magnitude improvement* from 1970 to 2001.

For other kinds of linear equations, the improvements indeed have been approximately the same..

fish

13 September 2011 at 11:32

Ha, good to know re: matrix inversion — I often use `invert` in calculating color transforms (see http://www.brucelindbloom.com/index.html?Eqn_RGB_XYZ_Matrix.html) but hey. Thanks!

Juan Solano

13 September 2011 at 11:33

Let me recommend the videos in Khan Academy, is a very nice place to start and refresh the key concepts.. and then digest the Linear Algebra beauty...

<http://www.khanacademy.org/#browse>

Juan

J

13 September 2011 at 12:18

Charles Van Loan has a sweet pair of custom Nike kicks. Not even joking.

martin

13 September 2011 at 15:53

Could you elaborate on number 6? I would have thought it would have taken close to a millisecond to just read a million values.

mg

13 September 2011 at 17:07

@Evan my favorite text is Janich's Linear Algebra.

Evan

13 September 2011 at 17:16

Thanks mg. I'll check that out as well.

AV

14 September 2011 at 04:20

this article is a 'triviality'.

any indian schoolkid knows this.

– of course linear algebra is balzing fast for FEA and related computations.

but the writer should state that the 'speed' in fast lin algebra comes not from inherent properties of lin-algebra, but from the fact that we MASSIVELY approximate the actual behavior of the beams/trusses/etc. we approximate reality. and thats becoz we dont have good computational algos to handle the 'true' mathematical representations.

if that was 'overhead transmission' – then here's a simplification– try to find a 'fast' solution to the Navier Stokes equations using 'linear algebra' – in their pure form.

Anselmo

19 November 2011 at 20:48

For me, the greatest achievement of linear algebra is the simplex method. Linear optimisation problems with million variables may be solved in home computers in a few minutes. It is remarkably efficient, though it has an exponential worst case performance. In addition, most exact

non-linear optimisation algorithms use the simplex method for solving the linear approximation model.

babita

19 May 2012 at 09:02

i want to solve the equation $Ax=B$, where A is sparse given in CSC format. A and B are known.

Can you give me some references to solve it???

Reply as soon as possible...its urgent..

Federico Poloni

13 February 2013 at 12:49

"The heart of Google is an enormous linear algebra problem."

This is often heard, and the algorithm itself is a nice one to tell around, but I think its role in the architecture of a search engine is overrated.

1- While PageRank constitutes a part of the algorithm that is used to rank sites (allegedly – it's not like it's open source), there must be much more than that. PageRank returns a universal ranking, that does not depend on the query. Clearly the ordering has to be tweaked to the specific search. Pagerank is not computed on-the-fly for every web search, it would be too slow to do that.

2- Ranking isn't the only part of the problem. There is crawling, indexing, storing data, and recalling results efficiently. PageRank might be a nice idea that constitutes 1% of that, but the bulk of the computation is many boring details on storing and retrieving compressed data.

John

13 February 2013 at 13:37

Frederico: The basic idea that made Google a better search engine than competitors was an eigenvalue problem. That's not to say that the algorithm hasn't been tweaked, or that it's easy to implement, or that many other factors don't matter.

Imran

7 January 2023 at 18:27

What about compressed sensing? I think that's quite an amazing application of matrix methods

Comments are closed.

Search ...

Search



John D. Cook, PhD

My colleagues and I have decades of consulting experience helping companies solve complex problems involving data privacy, applied math, and statistics.

Let's talk. We look forward to exploring the opportunity to help your company too.

JOHN D. COOK

© All rights reserved.

Search ...

SEARCH

(832) 422-8646

EMAIL